

# AutoSolver 设计说明报告

router

## 1. 摘要

AutoSolver 是一个面向配送任务分配的“多策略自动求解器”。它把每一条候选分配都看成一次有成本、有成功率、有约束的业务选择，然后在 10 秒左右的比赛时间内，用一组互补策略快速试探、精确搜索和局部修复，选出期望成本最低的合法方案。

可以把它理解成一个配送调度主管：

- 先快速看一遍“便宜且靠谱”的候选。
- 再识别“一个骑手带走多个订单”的合单机会。
- 对小规模场景尝试更精细的全局搜索。
- 对已有方案继续加固，例如给同一个任务包补充备用骑手，提高最终接单概率。
- 每一步都用同一套评分器验算，只有真正更好的方案才会被保留。

## 2. 业务问题不是简单派单

输入数据中的每一行表示一个“候选分配”：

| task id list | courier id | total score | willingness |
|--------------|------------|-------------|-------------|
| T0001        | C001       | 10.0        | 0.50        |
| T0001        | C002       | 12.0        | 0.90        |
| T0001,T0002  | C003       | 18.0        | 0.80        |

普通派单系统可能只会问：“哪个骑手分数最低？”本项目真正要回答的是：“在骑手可能拒单、订单不能重复归属、骑手不能重复使用、合单候选已经固定的前提下，哪组选择的期望代价最低？”

这使问题有三个关键难点：

| 难点     | 直观含义                            | 设计应对            |
|--------|---------------------------------|-----------------|
| 骑手会拒单  | 分给骑手不等于订单一定完成                   | 用接单概率计算期望成本     |
| 订单不能重叠 | T0001,T0002 与 T0001,T0003 不能同时选 | 评估器和策略同时检查任务冲突  |
| 同包可多骑手 | 同一个任务包可以派给多个骑手尝试                | 评估器按联合成功率计算     |
| 时间很短   | 比赛单例约 10 秒                      | 多策略按预算顺序执行，随时可停 |

3. 评分逻辑的直观解释

每个任务包的成本由两部分组成。

- 任务包成本 = 成功概率 \* 预估配送分数 + 失败概率 \* 未完成罚分
- 未完成罚分 = 100 \* 任务包内订单数
- 总分 = 所有已选任务包成本之和 + 完全未分配订单数 \* 100

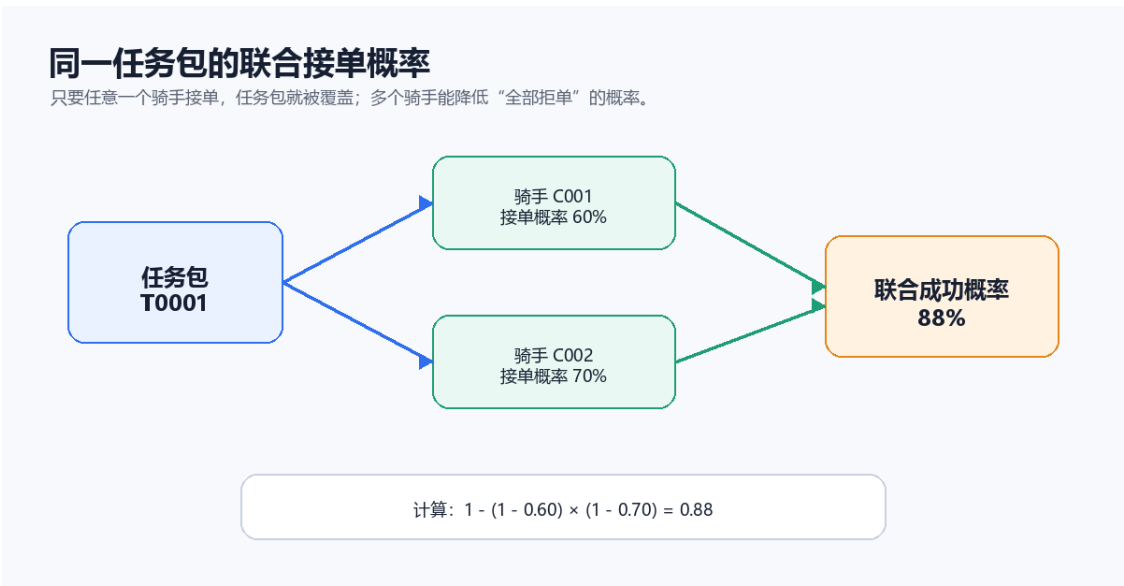
如果同一个任务包分配给多个骑手，成功概率会提升： $p\_complete = 1 - (1 - p1) * (1 - p2) * \dots * (1 - pn)$

例如，任务 T0001 分给两个骑手：

| 骑手   | 接单概率 |
|------|------|
| C001 | 0.60 |
| C002 | 0.70 |

联合成功概率不是 0.60 + 0.70，而是： $1 - (1 - 0.60) * (1 - 0.70) = 0.88$

也就是说，两个骑手都拒单的概率只有 12%。这就是 AutoSolver 会在合适情况下给同一个任务包补充骑手的原因：它相当于给关键订单加了一层“接单保险”。



同一任务包的联合接单概率

4. 整体架构

AutoSolver 的架构刻意保持简单：入口薄、模型轻、评分统一、策略可插拔。

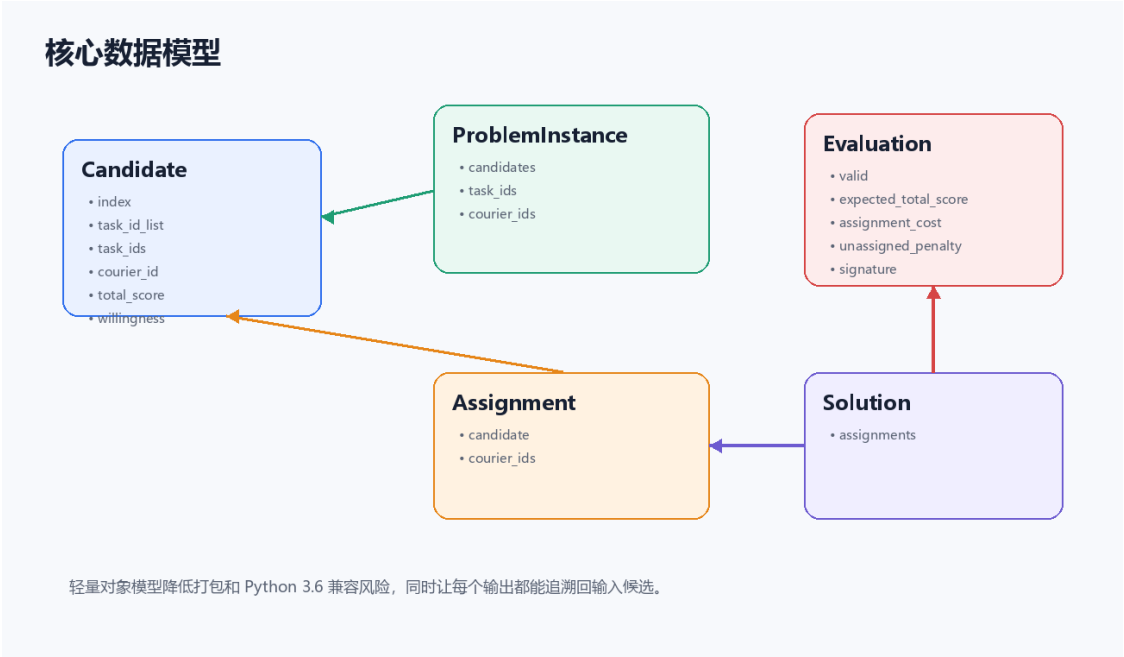


AutoSolver 主流程

核心模块职责如下：

| 模块                                   | 角色                                     | 设计价值                 |
|--------------------------------------|----------------------------------------|----------------------|
| <code>solver.py</code>               | 比赛入口，暴露 <code>solve(input_text)</code> | 对评测系统保持最小接口          |
| <code>autosolver/parser.py</code>    | 解析 TSV、跳过坏行、去除重复<br>输出键                | 输入越脏，越需要入口稳定         |
| <code>autosolver/models.py</code>    | 轻量对象模型                                 | 兼容 Python 3.6 和单文件打包 |
| <code>autosolver/evaluator.py</code> | 唯一评分口径                                 | 避免每个策略各算各的           |
| <code>autosolver/selector.py</code>  | 策略编排                                   | 让策略组合可控、确定、可复现       |
| <code>autosolver/strategies/</code>  | 多种求解策略                                 | 覆盖不同数据形态             |
| <code>pack.py</code>                 | 单文件打包                                  | 适配比赛提交环境             |

5. 数据模型视图



核心数据模型

这里有一个重要取舍：模型没有做复杂继承，也没有依赖第三方库。原因是提交环境要求 Python 3.6、标准库、单文件脚本。轻量模型让开发版和提交版之间的差异尽量小，降低“本地能跑、线上不能跑”的风险。

## 6. 多策略组合：不是一招鲜，而是策略车队

当前默认策略顺序来自 autosolver/selector.py:



默认策略流水线

这 13 个策略不是简单堆数量，而是分工不同：

| 策略组  | 代表策略                                                                      | 解决什么问题             |
|------|---------------------------------------------------------------------------|--------------------|
| 快速贪心 | GreedyByScore,<br>GreedyByExpectedScore,<br>GreedyByCoverage              | 快速得到稳定可用的初始解       |
| 覆盖感知 | GreedyCoverageAware,<br>PressureCoverageGreedy                            | 优先处理稀缺、昂贵、容易漏掉的任务  |
| 全局匹配 | SingletonMatchingGreedy,<br>SingletonBeamReassignment                     | 单任务场景下避免局部贪心抢错骑手   |
| 合单搜索 | BeamSetPackingSearch                                                      | 在候选合单之间寻找不重叠组合     |
| 限时精确 | ExactBranchAndBound                                                       | 小规模或可剪枝场景下逼近精确最优   |
| 加固修复 | MarginalSavingsGreedy,<br>ReinforceGreedy, PairSwapRepair,<br>LocalRepair | 对已有解补骑手、换骑手、替换局部选择 |

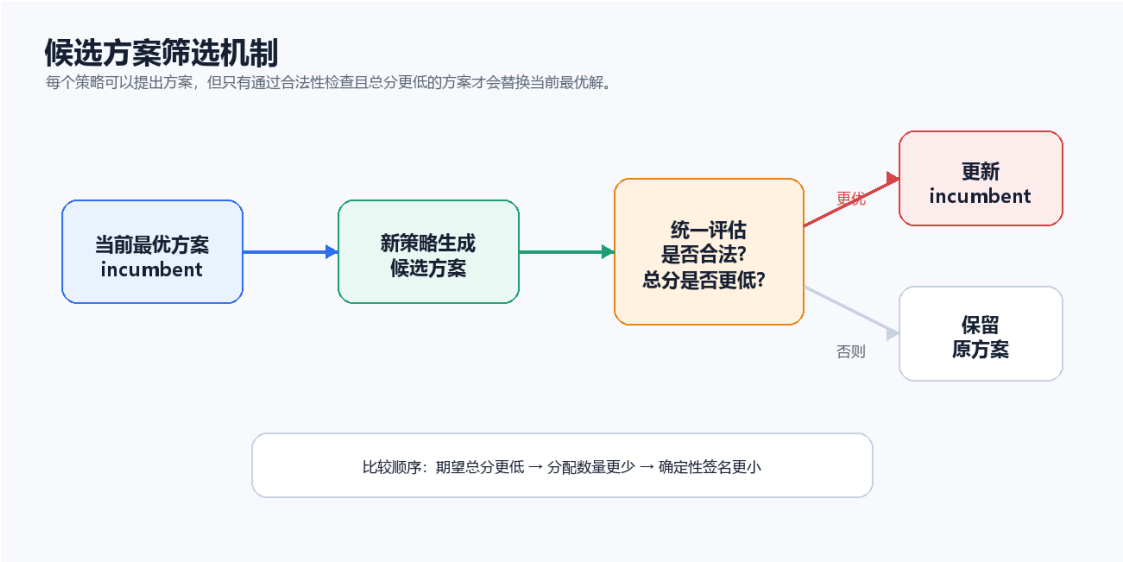
### 为什么要这样组合

不同测试用例的形态差异很大：

- 骑手充足时，重点是低成本和高成功率。
- 骑手稀缺时，重点是合单覆盖更多订单。

- 单任务包居多时，任务和骑手更像二分图匹配。
- 多任务包居多时，更像集合打包问题。
- 接单概率低时，给同包增加骑手可能比换一个低分骑手更重要。

因此，AutoSolver 不押注单一算法，而是让多种策略在同一评分器下竞争。谁能降低总分，谁就成为新的 incumbent。



候选方案筛选机制

## 7. 一个非算法视角的例子

假设有两个订单 T0001、T0002，系统有三个候选：

| 候选                  | 解释         | 风险           |
|---------------------|------------|--------------|
| T0001 -> C001       | 单独派一个订单    | 只覆盖一个订单      |
| T0002 -> C002       | 单独派另一个订单   | 需要占用第二个骑手    |
| T0001,T0002 -> C003 | 一个骑手合送两个订单 | 如果拒单，两个订单都失败 |

一个朴素算法可能只看 `total_score`，看谁低选谁。AutoSolver 会同时考虑：

- 合单是否减少骑手占用。
- 骑手接单概率是否足够高。
- 拒单后罚分是否太大。
- 这个选择会不会挡住后面更好的任务包。

所以它不会简单地说“合单一定好”或“最低分一定好”。它会把每个选择放入同一个期望成本公式中衡量，这也是设计稳定性的来源。

## 8. 关键设计决策

### 8.1 统一评分，而不是策略自说自话

所有策略最后都必须经过 `evaluate_solution()`。评估器会检查：

- 是否选择了未知候选。
- 同一订单是否进入了不同任务包。
- 同一骑手是否重复使用。
- 同任务包多骑手时的联合成功率。
- 未分配订单罚分。

比较方案时，当前源码使用如下优先级：

1. 期望总分更低。
2. 总分相同，则分配数量更少。
3. 仍相同，则使用确定性签名做字典序比较。

这个顺序让系统目标非常清晰：先控制业务成本，再减少冗余输出，最后保证结果可复现。

## 8.2 时间预算是核心边界

AutoSolver(time\_limit\_seconds=9.5) 使用 TimeBudget 控制整体运行。策略内部也会检查预算，尤其是搜索类策略：

- ExactBranchAndBound 默认最多使用 0.5 秒，且不超过剩余预算的一部分。
- PairSwapRepair 默认最多局部交换 2 秒。
- 束搜索和压力贪心会对任务数、骑手数、候选数做规模门控。

这保证了系统即使遇到大规模输入，也会优先返回已知最优方案，而不是为了某个慢策略拖垮整次提交。

## 8.3 提交环境友好

项目开发版虽然按模块组织，但提交版需要单文件。pack.py 负责把模块按模板展开，并尽量去掉类型提示和 typing 导入，以满足比赛环境：

| 约束         | 对应设计                        |
|------------|-----------------------------|
| Python 3.6 | 避免新语法，打包时去类型提示              |
| 标准库        | 求解器核心不依赖第三方包                |
| 单文件提交      | pack.py 生成 dist/solver_*.py |
| 10 秒限制     | 策略顺序 + 时间预算 + 规模门控          |

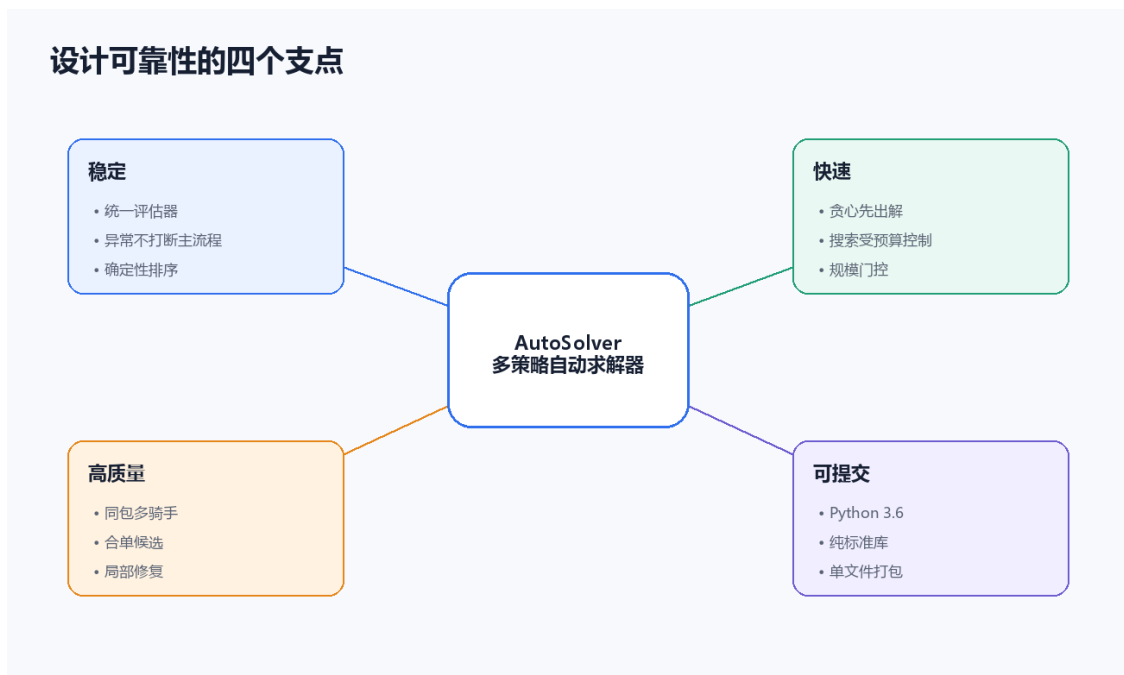
## 8.4 离线参考解用于校准方向

开发过程中设计了独立的现代 Python 离线求解环境，可以使用 Gurobi 求解器和列生成算法生成近优参考解。它不进入比赛提交文件，而是用于判断当前启发式策略距离高质量解还有多远。

这种分层很重要：

- 线上提交：轻、快、稳，只使用标准库。
- 离线参考：重、慢、强，用来提供质量标尺。

## 9. 设计可靠性



设计可靠性的四个支点

可靠性来自四个层面：

1. **业务口径可靠**：评分公式直接体现拒单风险和未分配罚分。
2. **工程边界可靠**：所有策略都受时间预算约束，主循环捕获策略异常。
3. **结果可解释**：每个输出都能还原到输入候选，且可通过评估器重新计算成本。
4. **演进空间明确**：新增策略只需实现统一接口，不需要重写主流程。